1 Write a Python program to implement single and multilevel inheritance

and show method overriding.

AIM

To write a Python program that implements single inheritance, multilevel inheritance, and demonstrates method overriding.

ALGORITHM

1. Start the program.
2. Create a base class () with a method .
3. Create a derived class () that inherits from (single inheritance).
4. Create another class () that inherits from (multilevel inheritance).
5. Override the method in derived classes to show different outputs.
6. Create objects of each class and call the method.
7. End program.

CODE

```
# Base Class
class Person:
    def __init__(self, name):
        self.name = name

    def display(self):
        print(f"Person Name: {self.name}")



# Single Inheritance: Student inherits from Person
class Student(Person):
    def __init__(self, name, roll_no):
        super().__init__(name)
```

```
        self.roll_no = roll_no

    # Method Overriding
    def display(self):
        print(f"Student Name: {self.name}, Roll No: {self.roll_no}")


# Multilevel Inheritance: GraduateStudent inherits from Student
class GraduateStudent(Student):
    def __init__(self, name, roll_no, course):
        super().__init__(name, roll_no)
        self.course = course

    # Method Overriding
    def display(self):
        print(f"Graduate Student Name: {self.name}, Roll No: {self.roll_no}, Course: {self.course}")


# Object Creation
p = Person("Yogin")
s = Student("Amit", 101)
g = GraduateStudent("Priya", 202, "Computer Science")

# Display details
print("=== Single Inheritance Example ===")
p.display()
s.display()
```

```
print("\n=== Multilevel Inheritance Example ===")
g.display()
```

OUTPUT

```
=== Single Inheritance Example ===
Person Name: Yogin
Student Name: Amit, Roll No: 101

=== Multilevel Inheritance Example ===
Graduate Student Name: Priya, Roll No: 202, Course: Computer Science
```

CONCLUSION

This program demonstrates:

- Single Inheritance → inherits from .
- Multilevel Inheritance → inherits from .
- Method Overriding → Each class redefines the method to provide specialized behavior.

👉 This shows how inheritance promotes code reuse and method overriding allows customization in Object-Oriented Programming.

2 Write a program to demonstrate simple single inheritance.

AIM

To write a Python program that demonstrates single inheritance, where a derived class inherits properties and methods from a base class.

ALGORITHM

1. Start the program.
2. Define a base class () with attributes and a method.
3. Define a derived class () that inherits from .

4. Add extra attributes/methods in the derived class.
5. Create objects of both classes.
6. Call methods to show inheritance.
7. End program.

CODE

```
# Base Class
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")



# Derived Class (Single Inheritance)
class Student(Person):
    def __init__(self, name, age, roll_no):
        # Call constructor of base class
        super().__init__(name, age)
        self.roll_no = roll_no

    def show(self):
        print(f"Student Roll No: {self.roll_no}")



# Object Creation
p = Person("Yogin", 20)
```

```
s = Student("Amit", 22, 101)

# Display details
print("=== Person Details ===")
p.display()

print("\n=== Student Details (Inherited + Own) ===")
s.display()   # inherited method
s.show()      # derived class method
```

OUTPUT

```
=== Person Details ===
Name: Yogin, Age: 20

=== Student Details (Inherited + Own) ===
Name: Amit, Age: 22
Student Roll No: 101
```

CONCLUSION

This program demonstrates:

- Single Inheritance → inherits from .
- Constructor chaining using .
- Method reuse → uses from .
- Extended functionality → adds its own method .

👉 This shows how inheritance allows code reuse and extension in Object-Oriented Programming.